

RabbitMQ {

<Por="Grupo 1"/>

}



Contenidos

- 01 RabbitMQ
- 02 RabbitMQ VS RMI
- 03 HelloRabbit
- 04 WorkQueues java
- 05 WorkQueues ruby
- 06 Ejemplos
- 07 Conclusión

RabbitMQ {

RabbitMQ es un broker de mensajes (intermediario), es decir.

Un sistema independiente que permite que diferentes aplicaciones se comuniquen entre sí enviando mensajes.

En vez de que dos programas se conecten directamente, usan RabbitMQ como intermediario.



Contruido por



Erlang creado para manejar millones de conexiones concurrentes sin caerse nunca



Erlagn en su Backend para manejar miles de millones de mensajes sin colapsar

}

RabbitMQ vs RMI {

Un programa (CLIENTE) llama a un método (SERVIDOR)
y se queda esperando en la línea hasta que le
responden.

RMI (COMUNICACIÓN SÍNCRONA)



PROBLEMA

- Acoplados.
- Ambos tienen que estar vivos y conectados al mismo tiempo.

}

RabbitMQ vs RMI {

“A diferencia de RMI, donde un programa se conecta directamente a otro para ejecutar métodos, RabbitMQ no requiere una conexión directa.

El productor simplemente envía el mensaje a un intermediario, RabbitMQ, que actúa como una oficina central de correos, encargándose de distribuir los mensajes a los consumidores.”

RABBITMQ (COMUNICACIÓN ASÍNCRONA)



VENTAJA

- Si el programa receptor está apagado, el sistema no falla.
- RabbitMQ almacena los mensajes en una cola y los entrega cuando el receptor vuelve a estar disponible

}

Instalaciones RabbitMQ y Ruby {

```
rabbitmq-plugins enable rabbitmq_management
```

```
rabbitmq-plugins enable rabbitmq_stream
```

```
.\rabbitmq-service start
```

<http://localhost:15672>

guest



rubyinstaller-de
vkit-3.4.9-1-x64
.exe

```
ruby worker.rb
```

```
ruby new_task.rb
```

}

HelloRabbit {

Send (PRODUTOR)

```
public class Send {  
    private final static String QUEUE_NAME = "hello";  
  
    public static void main(String[] argv) throws Exception {  
        ConnectionFactory factory = new ConnectionFactory();  
        factory.setHost("localhost");  
  
        try (Connection connection = factory.newConnection();  
            Channel channel = connection.createChannel()) {  
  
            channel.queueDeclare(QUEUE_NAME, false, false, false, null);  
            String message = "Hello World!";  
  
            channel.basicPublish("", QUEUE_NAME, null, message.getBytes("UTF-8"));  
            System.out.println(" [x] Sent " + message + "");  
  
        }  
    }  
}
```

- Send se conecta a RabbitMQ
- Declara una cola ("hello")
- Envía un mensaje a la cola

Receive (Consumidor)

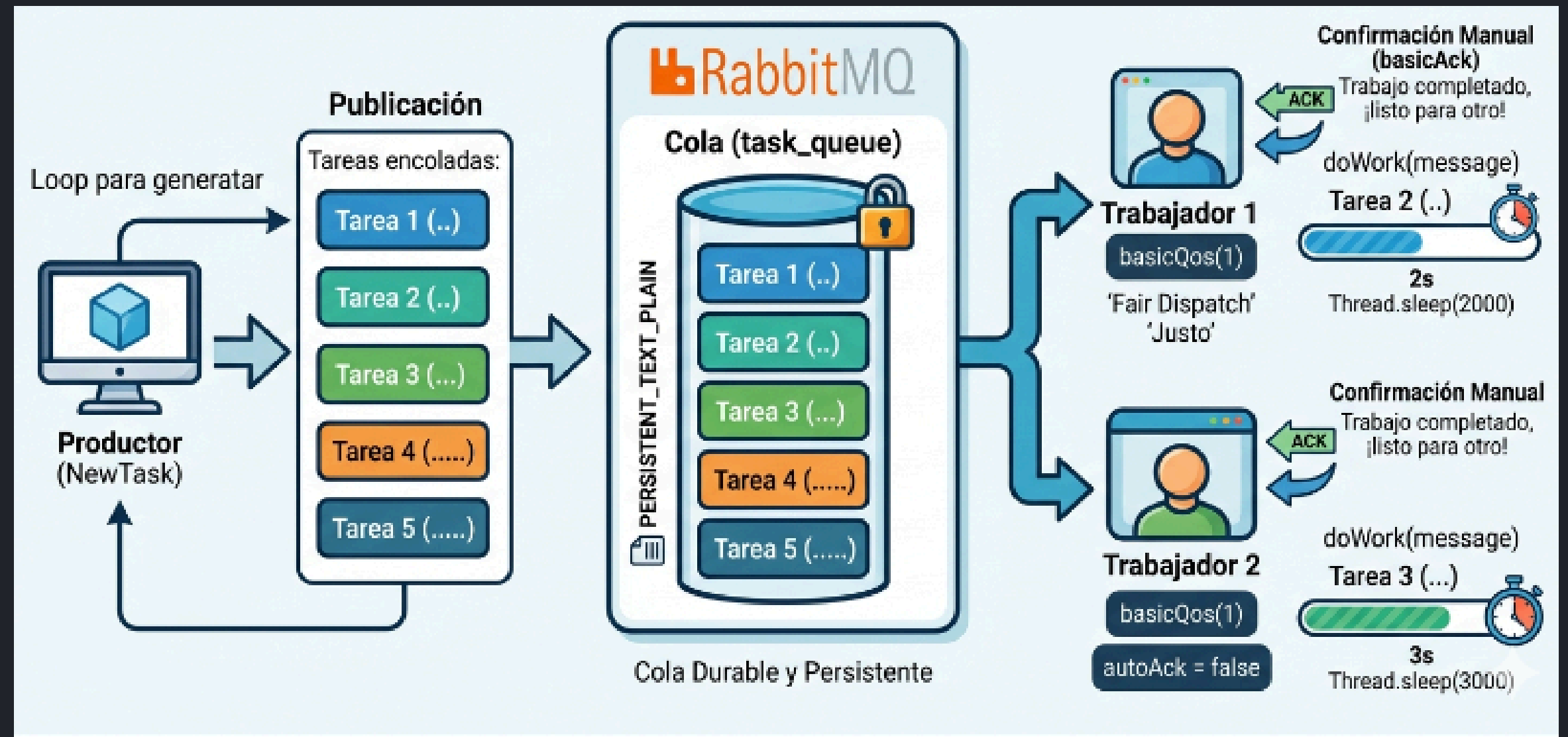
```
public class Receive {  
    private final static String QUEUE_NAME = "hello";  
  
    public static void main(String[] argv) throws Exception {  
        ConnectionFactory factory = new ConnectionFactory();  
        factory.setHost("localhost");  
  
        Connection connection = factory.newConnection();  
        Channel channel = connection.createChannel();  
  
        channel.queueDeclare(QUEUE_NAME, false, false, false, null);  
        System.out.println(" [*] Waiting for messages. To exit press CTRL+C");  
  
        DeliverCallback deliverCallback = (consumerTag, delivery) -> {  
            String message = new String(delivery.getBody(), "UTF-8");  
            System.out.println(" [x] Received '" + message + "'");  
        };  
  
        channel.basicConsume(QUEUE_NAME, true, deliverCallback, consumerTag -> {});  
    }  
}
```

- Se conecta a RabbitMQ
- Escucha la cola ("hello")
- Recibe y muestra los mensajes

}

Work queues

- Cola durable
- Mensajes persistentes
- ACK manual



- Limita a n tareas por worker

Cola durable

```
// durable = true (La cola no se borra si se reinicia RabbitMQ)  
channel.queueDeclare(TASK_QUEUE_NAME, true, false, false, null);
```

Mensajes persistentes

```
// PERSISTENT_TEXT_PLAIN (El mensaje se guarda en disco)  
channel.basicPublish("", TASK_QUEUE_NAME,  
    MessageProperties.PERSISTENT_TEXT_PLAIN,  
    message.getBytes("UTF-8"));
```

Control del trabajo

-ACK manual

```
// autoAck = false (Nosotros confirmamos manualmente)  
boolean autoAck = false;  
channel.basicConsume(TASK_QUEUE_NAME, autoAck, deliverCallback, consumerTag -> { });
```

-Limitar a 1 tarea por worker

```
// SORTEO JUSTO: RabbitMQ solo le dará 1 tarea a la vez a este trabajador  
channel.basicQos(1);
```

Ejemplo de trabajo NewTask.java

```
channel.basicPublish(exchange: "", routingKey: "task_queue",  
    MessageProperties.PERSISTENT_TEXT_PLAIN,  
    message.getBytes());
```

```
// Enviamos 5 tareas automáticamente para ver el sorteo  
for (int i = 1; i <= 5; i++) {  
    // Cada tarea tiene una cantidad diferente de puntos (segundos de trabajo)  
    String message = "Tarea " + i + " " + ".".repeat(i);
```

```
    // PERSISTENT_TEXT_PLAIN (El mensaje se guarda en disco)  
    channel.basicPublish("", TASK_QUEUE_NAME,  
        MessageProperties.PERSISTENT_TEXT_PLAIN,  
        message.getBytes("UTF-8"));
```

```
    System.out.println(" [x] Enviado: '" + message + "'");
```

```
}
```

Worker.java

Función que actúa cuando recibe un mensaje

```
DeliverCallback deliverCallback = (consumerTag, delivery) -> {  
    String message = new String(delivery.getBody(), "UTF-8");  
    System.out.println(" [x] Recibida: '" + message + "'");  
  
    try {  
        doWork(message);  
    } finally {  
        System.out.println(" [x] Terminado");  
        // Confirmación manual (Avisamos que ya estamos libres para otra tarea)  
        channel.basicAck(delivery.getEnvelope().getDeliveryTag(), false);  
    }  
};
```

Función que actúa cuando recibe un mensaje

```
// Método que simula el trabajo: se pausa 1 segundo por cada punto "."
private static void doWork(String task) {
    for (char ch : task.toCharArray()) {
        if (ch == '.') {
            try {
                Thread.sleep(1000);
            } catch (InterruptedException _ignored) {
                Thread.currentThread().interrupt();
            }
        }
    }
}
```


Libreria de Ruby

Abrir terminal (power shell)

Escribimos el comando:

`gem install bunny`

```
PS C:\Users\TODO LAPTOP\Desktop\RabbitMQ\RabbitMQRuby> gem install bunny
Using rubygems directory: C:/Users/TODO LAPTOP/.local/share/gem/ruby/3.4.0
Building native extensions. This could take a while...
Successfully installed rbtrees-0.4.6
Successfully installed sorted_set-1.1.0
Successfully installed logger-1.7.0
Successfully installed amqp-protocol-2.7.0
Successfully installed bunny-3.1.0
5 gems installed

A new release of RubyGems is available: 3.6.9 → 4.0.10!
Run `gem update --system 4.0.10` to update your installation.
```

Carpeta Ruby

Nombre	Fecha de modificación	Tipo
 new_task.rb	4/12/2026 9:39 PM	Ruby File
 worker.rb	4/16/2026 3:47 PM	Ruby File

Productor (new_task.rb) → envía tareas
Consumidor (worker.rb) → recibe y ejecuta
tareas

Ruby - New Task

```
1  require 'bunny'
2
3  # Conectarse a RabbitMQ
4  connection = Bunny.new(hostname: 'localhost')
5  connection.start
6
7  channel = connection.create_channel
8  # Declarar la cola como durable (no se borra al reiniciar)
9  queue = channel.queue('task_queue', durable: true)
10
11  puts "Enviando 5 tareas a la cola..."
```

```
12
13  (1..5).each do |i|
14    # Creamos el mensaje con puntos que representan segundos
15    dots = "." * i
16    message = "Tarea #{i} #{dots}"
17
18    # Enviar el mensaje marcándolo como persistente
19    queue.publish(message, persistent: true)
20    puts " [x] Enviado: '#{message}'"
21  end
22
```

```
Windows PowerShell

Instale la versión más reciente de PowerShell para obtener nuevas caract
erísticas y mejoras. https://aka.ms/PSWindows

PS C:\Cdkav04\6to Semestre\SIS258 - SISTEMAS DISTRIBUIDOS 2026\SistemasD
istribuidosCDK 2026\Practicas\RabbitMQ - Grupo 1\RabbitMQRuby> ruby work
er.rb
[*] Esperando tareas. Para salir presiona CTRL+C
[x] Recibida: 'Tarea 5 .....
```

```
Windows PowerShell

Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas caract
erísticas y mejoras. https://aka.ms/PSWindows

PS C:\Cdkav04\6to Semestre\SIS258 - SISTEMAS DISTRIBUIDOS 2026\SistemasD
istribuidosCDK 2026\Practicas\RabbitMQ - Grupo 1\RabbitMQRuby> ruby work
er.rb
[*] Esperando tareas. Para salir presiona CTRL+C
[x] Recibida: 'Tarea 3 ...'
[x] Terminado
```

```
Windows PowerShell

Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas caract
erísticas y mejoras. https://aka.ms/PSWindows

PS C:\Cdkav04\6to Semestre\SIS258 - SISTEMAS DISTRIBUIDOS 2026\SistemasD
istribuidosCDK 2026\Practicas\RabbitMQ - Grupo 1\RabbitMQRuby> ruby wor
ker.rb
[*] Esperando tareas. Para salir presiona CTRL+C
[x] Recibida: 'Tarea 2 ...'
[x] Terminado
[x] Recibida: 'Tarea 5 .....
```

```
Windows PowerShell

Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas caract
erísticas y mejoras. https://aka.ms/PSWindows

PS C:\Cdkav04\6to Semestre\SIS258 - SISTEMAS DISTRIBUIDOS 2026\SistemasD
istribuidosCDK 2026\Practicas\RabbitMQ - Grupo 1\RabbitMQRuby> ruby new_
task.rb
Enviando 5 tareas a la cola...
[x] Enviado: 'Tarea 1 ...'
[x] Enviado: 'Tarea 2 ...'
[x] Enviado: 'Tarea 3 ...'
[x] Enviado: 'Tarea 4 ...'
[x] Enviado: 'Tarea 5 .....
```

```
<!--Estudio Shonos-->
```

Gracias {

```
<Por="Grupo 1"/>
```

}